ByteMart Digital

ACE THE QA INTERVIEW

Manual Testing

Interview Questions & Answers

The questions that actually get asked — concept and coding — each with a clear, correct answer, worked code, and the gotchas interviewers look for.

120+	Concept	10	All Levels
Questions	+ Coding	Topic Areas	Difficulty

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Manual Testing: Interview Questions & Answers

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors or omissions. All code examples are provided "as is". Interview questions and expected answers vary by company and role.

Trademarks
Python is a registered trademark of the Python Software Foundation. All other names and brands are the property of their respective owners.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. Testing Fundamentals 4

2. SDLC & STLC 8

3. Testing Levels & Types 12

4. Test Design Techniques 16

5. Test Documentation & Process 20

6. Defect Management 24

7. Specialized Testing Types 28

8. API & Database Testing 32

9. Agile & Test Management 36

10. Scenario & Practical Questions 40

PART 01

Testing Fundamentals

The core definitions and distinctions every QA interview opens with.

13 questions

Q01 Basic

What is software testing?

Software testing is the process of evaluating an application to find defects and verify that it meets its requirements. It builds confidence that the software works correctly and behaves as users expect — before it reaches them.

Q02 Basic

What is the difference between testing and debugging?

Testing finds defects — it reveals that something is wrong. **Debugging** is what developers do afterward to locate and fix the cause. Testers report the symptom; developers cure it.

Q03 Intermediate

What is the difference between QA and QC?

Quality Assurance	Quality Control
process-oriented	product-oriented
preventive — builds quality in	detective — finds defects
the whole process	testing the product

Testing is a QC activity within a broader QA process.

Q04 Intermediate

What is the difference between verification and validation?

- **Verification** — 'are we building it right?' Static checks: reviews, walkthroughs, inspections.
- **Validation** — 'are we building the right thing?' Dynamic checks: actually running the software.

Q05 Intermediate

What is the difference between error, defect, bug, and failure?

- **Error** — a human mistake in code or design.
- **Defect / Bug** — the flaw in the software that results.
- **Failure** — the defect manifesting during execution.

© 2026 ByteMart Digital

An error leads to a defect, which — when executed under the right conditions — causes a failure.

Q06 Intermediate

What are the seven principles of testing?

- Testing shows the presence of defects, not their absence.
- Exhaustive testing is impossible.
- Early testing saves time and money.
- Defects cluster together.
- Beware the pesticide paradox — vary your tests.
- Testing is context-dependent.
- Absence-of-errors is a fallacy.

Q07 Intermediate

Why can't we test a program completely?

Exhaustive testing is impossible — the number of possible inputs, paths, and combinations is effectively infinite. Instead, we use risk analysis and techniques like equivalence partitioning to test the most valuable subset.

Q08 Advanced

What is the pesticide paradox?

Running the same tests repeatedly eventually stops finding new defects — like a pesticide insects grow immune to. The fix is to regularly review and update test cases and explore untested areas.

Q09 Basic

What is the difference between a test case and a test scenario?

A **test scenario** is a high-level description of what to test ('verify login'). A **test case** is the detailed set of steps, data, and expected results that verifies it. One scenario yields several test cases.

Q10 Basic

What does 'severity' mean in testing?

Severity is the technical impact of a defect on the system — how badly it breaks functionality. It's set by the tester, independent of how urgently the business wants it fixed (priority).

© 2026 ByteMart Digital

Q11 Intermediate

What is the difference between static and dynamic testing?

Static	Dynamic
no code execution	code is executed
reviews, inspections	running test cases
finds defects early	finds runtime defects

Q12 Basic

What is positive and negative testing?

- **Positive testing** — valid input works as expected (correct password logs in).
- **Negative testing** — invalid input is handled gracefully (wrong password shows a clear error).

Q13 Advanced

What is the 'absence-of-errors' fallacy?

It's the mistake of thinking a bug-free product is automatically a good one. If the software passes every test but doesn't meet the user's actual needs, it has still failed — quality means fitness for purpose, not just the absence of defects.

© 2026 ByteMart Digital

PART 02

SDLC & STLC

How testing fits into the development and testing life cycles.

12 questions

02

Q01 Basic

What is the SDLC?

The **Software Development Life Cycle** describes the phases of building software: requirements, design, development, testing, deployment, and maintenance. Models like Waterfall, V-Model, and Agile arrange these phases differently.

Q02 Basic

What is the STLC?

The **Software Testing Life Cycle** is the structured sequence of testing phases: requirement analysis, test planning, test case design, environment setup, test execution, and test cycle closure — each with entry and exit criteria.

Q03 Intermediate

What is the V-Model?

The **V-Model** pairs each development phase with a corresponding testing phase — requirements with acceptance testing, design with system/integration testing, coding with unit testing. Test planning starts early, in parallel with development.

Q04 Intermediate

What is the difference between the Waterfall and Agile models?

Waterfall	Agile
sequential phases	iterative sprints
testing is a late phase	testing every sprint
heavy documentation	lightweight docs
rigid to change	embraces change

Q05 Intermediate

What are entry and exit criteria?

- **Entry criteria** — conditions that must be met to start a phase (e.g. stable build, cases reviewed).
- **Exit criteria** — conditions that must be met to complete it (e.g. 95% pass, no open critical defects).

© 2026 ByteMart Digital

Q06 Intermediate

What are the phases of the STLC?

- **Requirement analysis** — understand what to test.
- **Test planning** — strategy, scope, effort.
- **Test case design** — write cases and data.
- **Environment setup** — prepare the test environment.
- **Test execution** — run cases, log defects.
- **Cycle closure** — report and review.

Q07 Intermediate

What is a Requirements Traceability Matrix (RTM)?

An **RTM** maps each requirement to the test cases that verify it. It proves coverage — every requirement has a test — and reveals gaps where requirements are untested.

Q08 Intermediate

When does testing start in a project?

Testing should start as early as the **requirements phase** — reviewing requirements for clarity and testability. This 'shift left' catches defects when they're cheapest to fix, long before code is written.

Q09 Intermediate

What is the cost of fixing a defect over time?

The cost rises dramatically the later a defect is found — roughly 1x in requirements, ~10x in testing, and 30–100x in production. This is the core economic argument for early testing.

Q10 Advanced

What is the difference between a test plan and a test strategy?

A **test strategy** is a high-level, often organisation-wide document defining the overall testing approach. A **test plan** is project-specific — detailing scope, resources, schedule, and criteria for that particular effort.

© 2026 ByteMart Digital

Q11 Advanced

What is meant by 'shift-left' testing?

Shift-left means moving testing activities earlier in the life cycle — involving testers from requirements onward rather than only at the end. It catches defects sooner, when they're far cheaper to fix.

Q12 Advanced

Which SDLC model is best for testing?

There's no single best — it depends on the project. Agile suits evolving requirements and fast feedback; the V-Model suits stable, well-defined requirements needing rigour (e.g. medical, aviation). The right choice matches the project's needs and risk.

© 2026 ByteMart Digital

PART 03

Testing Levels & Types

The levels software is tested at, and the many types of testing.

13 questions

Q01 Basic

What are the levels of testing?

Level	Tests	By
Unit	individual components	developers
Integration	combined modules	devs / testers
System	the complete system	testers
Acceptance	against user needs	users

Q02 Intermediate

What is the difference between functional and non-functional testing?

- **Functional** — what the system does (login works, cart calculates correctly).
- **Non-functional** — how well it does it (performance, security, usability, compatibility).

Q03 Intermediate

What is the difference between black box, white box, and grey box testing?

Approach	Code knowledge
Black box	none — tests behaviour
Grey box	partial
White box	full — tests internal logic

Q04 Intermediate

What is the difference between smoke and sanity testing?

Smoke	Sanity
broad & shallow	narrow & deep
whole build stability	a specific fix/area
often scripted	usually unscripted

© 2026 ByteMart Digital

Q05 Basic

What is regression testing?

Regression testing re-runs existing tests after a change to confirm that new code hasn't broken previously working features. It's the safety net of every release and the area where automation pays off most.

Q06 Intermediate

What is the difference between retesting and regression testing?

- **Retesting** — verifies a specific fixed defect is now resolved (planned, same test).
- **Regression** — checks the fix didn't break anything else (broader, other areas).

Q07 Intermediate

What is integration testing, and what are its approaches?

Integration testing verifies that combined modules work together. Approaches: **Big Bang** (all at once), **Top-down** (high-level first, using stubs), **Bottom-up** (low-level first, using drivers), and **Sandwich** (both).

Q08 Intermediate

What is the difference between alpha and beta testing?

Alpha	Beta
in-house testers	real users
controlled environment	real environment
before beta	before final release

Q09 Basic

What is User Acceptance Testing (UAT)?

UAT is the final testing level where real users confirm the software meets their needs in realistic scenarios. It answers 'is this what we actually wanted?' — the last gate before release.

© 2026 ByteMart Digital

Q10 Advanced

What are stubs and drivers?

- **Stub** — a dummy that stands in for a not-yet-ready called module (used in top-down integration).
- **Driver** — a dummy that calls a module under test (used in bottom-up integration).

Q11 Basic

What is compatibility testing?

Compatibility testing verifies the software works consistently across different browsers, devices, operating systems, and screen sizes — focused on the combinations your real users actually have.

Q12 Intermediate

What is end-to-end testing?

End-to-end testing validates a complete workflow from start to finish, as a real user would experience it — across the UI, backend, database, and integrations — rather than testing components in isolation.

Q13 Intermediate

What is exploratory testing?

Exploratory testing is simultaneous learning, test design, and execution — the tester investigates the application freely, forming and following hypotheses in real time. It often finds defects scripted tests miss.

© 2026 ByteMart Digital

PART 04

Test Design Techniques

The techniques that turn 'test everything' into smart, minimal test sets.

12 questions

Q01 Intermediate

What is equivalence partitioning?

Equivalence Partitioning (EP) groups inputs into classes that should behave the same, then tests one representative per class. If a field accepts 1–100, testing 50 represents all valid values — dramatically cutting the number of tests.

Q02 Intermediate

What is boundary value analysis?

Boundary Value Analysis (BVA) tests the edges of each partition, where defects cluster. For a range 18–60, you test 17, 18, 19, 59, 60, 61 — the values just inside and outside the boundaries.

Q03 Advanced

Why do we use EP and BVA together?

EP reduces the total number of test cases by grouping inputs; **BVA** ensures the risky edges within those partitions are covered. Together they give strong coverage with a small, defensible set of tests.

Q04 Intermediate

What is a decision table, and when do you use it?

A **decision table** maps combinations of input conditions to expected actions. Use it when behaviour depends on several conditions together — like eligibility or pricing rules — so no combination is missed.

Member?	Cart>500?	Discount
Yes	Yes	20%
Yes	No	10%
No	No	0%

Q05 Intermediate

What is state transition testing?

State transition testing checks how a system behaves as it moves between states in response to events — like an order going draft → placed → shipped. You verify valid transitions work and invalid ones are blocked.

© 2026 ByteMart Digital

Q06 Basic

What is error guessing?

Error guessing is an experience-based technique where the tester uses intuition and past experience to guess where defects are likely — empty fields, zero, special characters, huge values — and targets them.

Q07 Basic

What is ad-hoc testing?

Ad-hoc testing is informal, unstructured testing without documentation — trying random inputs and unusual flows to 'break' the software, relying purely on tester creativity and experience.

Q08 Basic

What makes a good test case?

- **Clear** — unambiguous steps anyone can follow.
- **Atomic** — tests one thing, so failures are precise.
- **Repeatable** — same steps give the same result.
- **Traceable** — links back to a requirement.

Q09 Basic

What fields does a test case contain?

Test Case ID, precondition, test steps, test data, expected result, actual result, and status (Pass/Fail). A complete case leaves no ambiguity about what to do or what to expect.

Q10 Intermediate

What is the difference between EP and BVA in one line?

EP tests one value from the middle of each valid/invalid range; **BVA** tests the values at the edges of those ranges. EP reduces count; BVA targets where bugs live.

Q11 Advanced

How do you decide which test design technique to use?

- **Ranges / limits** → boundary value analysis.

- **Groups of inputs** → equivalence partitioning.
- **Combinations of conditions** → decision table.
- **State-driven behaviour** → state transition.

Q12 Intermediate

Why is negative testing important?

Real users make mistakes and enter invalid data. **Negative testing** — wrong types, empty fields, special characters, oversized inputs — catches how gracefully the system handles errors, and typically finds more real defects than happy-path testing.

PART 05

Test Documentation & Process

The documents and workflow that structure a testing effort.

12 questions

Q01 Basic

What is a test plan, and what does it contain?

A **test plan** is the blueprint for testing a project. It contains: scope (in/out), approach, resources, schedule, entry/exit criteria, and risks with mitigation.

Q02 Basic

What is the difference between a test scenario and a test case?

A **test scenario** is a high-level 'what to test' (verify checkout). A **test case** is the detailed steps, data, and expected results. One scenario expands into multiple test cases.

Q03 Basic

What is a test suite?

A **test suite** is a collection of test cases grouped for a purpose or feature — for example, a 'regression suite' or a 'login suite' — that are executed together.

Q04 Intermediate

What is test data, and why does it matter?

Test data is the input used to execute test cases. Good test data covers valid, invalid, and boundary values. Poor or shared test data is a leading cause of unreliable, hard-to-reproduce test results.

Q05 Intermediate

What is a Requirements Traceability Matrix used for?

An **RTM** maps requirements to test cases, proving every requirement is covered by at least one test and highlighting gaps. It answers the manager's question: 'how do we know we've tested everything?'

Q06 Intermediate

What is a test summary report?

A **test summary report** communicates the results of a test cycle: cases executed, pass/fail counts, open defects, coverage, and a recommendation on release readiness — turning raw results into a go/no-go decision.

© 2026 ByteMart Digital

Q07 Basic

What is the difference between test execution and test design?

Test design is writing the test cases (before execution); **test execution** is running them, comparing actual vs expected, and logging defects. Design creates the tests; execution runs them.

Q08 Intermediate

What are entry and exit criteria for test execution?

- **Entry** — stable build deployed, environment ready, test cases reviewed.
- **Exit** — planned cases executed, pass rate met, no open critical/high defects.

Q09 Advanced

What is a traceability gap?

A **traceability gap** is a requirement with no test case covering it, or a test case with no linked requirement. The RTM surfaces both — the first is a coverage risk, the second may be wasted or orphaned testing.

Q10 Advanced

How do you estimate testing effort?

Estimation is based on the number and complexity of test cases, the scope of features, past project data, and available resources. Techniques include work breakdown, three-point estimation, and analogy to similar past work.

Q11 Basic

What documents does a tester typically produce?

- Test plan and test strategy.
- Test scenarios and test cases.
- Requirements Traceability Matrix (RTM).
- Defect reports and the test summary report.

© 2026 ByteMart Digital

Q12

Basic

What is the difference between a test plan and a test scenario?

A **test plan** is the overall strategy document for the whole testing effort (scope, approach, schedule). A **test scenario** is a single high-level thing to test within that effort. The plan governs many scenarios.

PART 06

Defect Management

Tracking defects from discovery to closure — and prioritising them well.

13 questions

Q01 Basic

What is the defect life cycle?

The **defect life cycle** is the set of states a bug moves through: New → Assigned → Open → Fixed → Retest → Closed — or Reopened if the fix fails. It ensures no defect is lost or left in limbo.

Q02 Intermediate

What is the difference between severity and priority?

- **Severity** — technical impact of the defect. Set by the tester.
- **Priority** — business urgency to fix it. Set by the product owner / manager.

They're independent — a bug can be high in one and low in the other.

Q03 Intermediate

Give an example of high severity but low priority.

A crash in a rarely used, obscure feature — technically severe (the app crashes) but low priority because almost no one hits it, so the business can defer the fix.

Q04 Intermediate

Give an example of low severity but high priority.

A typo in the company logo or brand name on the homepage — technically trivial (low severity) but embarrassing and highly visible, so it must be fixed immediately (high priority).

Q05 Basic

What makes a good bug report?

- **Clear title** summarising the problem.
- **Exact steps** to reproduce.
- **Expected vs actual** result.
- **Environment** — browser, OS, build.
- **Evidence** — screenshots, logs.

© 2026 ByteMart Digital

Q06 Intermediate

What is the single most important part of a bug report?

Clear, reliable steps to reproduce. If a developer can't reproduce a defect, they can't fix it — everything else in the report supports that goal.

Q07 Intermediate

What is defect density?

Defect density is the number of defects per unit size of the software (per module, per KLOC, or per feature). It helps identify the most defect-prone areas so testing can focus there.

Q08 Advanced

What is defect leakage?

Defect leakage is the number of defects that escaped testing and were found in production. A rising trend is one of the most honest signals that the testing process needs strengthening.

Q09 Intermediate

What is the difference between a defect and an enhancement?

A **defect** is the software failing to meet a stated requirement. An **enhancement** is a request for new or changed behaviour beyond the requirements. Reporting an enhancement as a bug wastes triage effort.

Q10 Basic

What does it mean when a defect is marked 'deferred' or 'rejected'?

- **Deferred** — a valid defect, but scheduled to be fixed in a later release.
- **Rejected** — not considered a valid defect (works as designed, duplicate, or not reproducible).

Q11 Advanced

When should you stop testing?

- Exit criteria in the test plan are met.
- Test cases executed and pass rate achieved.
- No open critical or high-severity defects.

© 2026 ByteMart Digital

- Deadline or budget reached (a business decision).

Since exhaustive testing is impossible, stopping is a risk-based decision — not 'when all bugs are gone.'

Q12 Advanced

What is defect triage?

Defect triage is a meeting where the team reviews open defects and decides their priority, severity, and who fixes them by when. It balances severity, priority, effort, and risk to sequence the work sensibly.

Q13 Advanced

What is defect masking?

Defect masking is when one defect hides another — the first prevents the code path that would reveal the second from ever executing. Fixing the first can suddenly expose the previously hidden defect.

© 2026 ByteMart Digital

PART 07

Specialized Testing Types

Usability, compatibility, performance, security, and mobile testing.

12 questions

Q01 Basic

What is usability testing?

Usability testing evaluates how easy and intuitive the software is to use — can a new user complete key tasks without confusion? It focuses on the human experience, not just functional correctness.

Q02 Intermediate

What is accessibility testing?

Accessibility testing ensures the software is usable by people with disabilities — keyboard navigation, screen-reader support, colour contrast, and alt text. It follows standards like WCAG and is often a legal requirement.

Q03 Basic

What is cross-browser testing?

Cross-browser testing verifies the application's layout and functionality work consistently across different browsers (Chrome, Firefox, Safari, Edge), which render pages and support features differently.

Q04 Intermediate

What is performance testing?

Performance testing checks how the system behaves under load — response time, throughput, and stability. Types include load, stress, spike, and soak testing. It's usually driven by tools like JMeter.

Q05 Intermediate

What is the difference between load and stress testing?

Load testing	Stress testing
expected load	beyond normal load
'does it handle X users?'	'where does it break?'

Q06 Intermediate

What is security testing?

Security testing identifies vulnerabilities that could let attackers access data or disrupt the system — checking authentication, authorisation, input validation, encryption, and common flaws like SQL injection and XSS.

Q07 Intermediate

What is the difference between UI and usability testing?

- **UI testing** — do the visual elements display and work correctly? (layout, buttons, fonts).
- **Usability testing** — is the software easy and intuitive to use? (flow, clarity, effort).

Q08 Intermediate

What are the challenges unique to mobile testing?

- Device fragmentation — countless models and OS versions.
- Interruptions — calls, notifications, low battery.
- Network variability — wifi, 4G, 3G, offline.
- Gestures, sensors, and limited resources.

Q09 Advanced

What is localization vs internationalization testing?

- **Internationalization (i18n)** — does the app support adaptation to other languages/regions?
- **Localization (L10n)** — is it correctly adapted for a specific locale (language, currency, formats)?

Q10 Intermediate

What is compatibility testing vs cross-browser testing?

Cross-browser is a subset of compatibility testing focused on browsers. **Compatibility testing** is broader — also covering operating systems, devices, screen sizes, and network conditions.

© 2026 ByteMart Digital

Q11 Basic

What is GUI testing?

GUI (Graphical User Interface) testing verifies the visual elements of an application — layout, alignment, colours, fonts, images, buttons, and their behaviour — display and respond correctly across screens.

Q12 Advanced

How would you approach testing a feature with no documentation?

- Explore the feature to learn how it works (exploratory testing).
- Talk to developers, product owners, and users to gather intent.
- Compare with similar features or industry norms.
- Document assumptions and confirm them as you go.

PART 08

API & Database Testing

Testing the interfaces and data behind the UI — manually and with tools.

13 questions

Q01 Basic

What is API testing?

API testing verifies the application's interfaces directly — sending requests and checking responses — without the UI. It's faster and more stable than UI testing and can start before the front-end exists.

Q02 Intermediate

Why test APIs directly instead of through the UI?

- Faster and more stable — no browser flakiness.
- Can test before the UI is built.
- Tests logic, validation, and error handling directly.
- Easily covers edge cases and negative scenarios.

Q03 Basic

What are the common HTTP methods?

Method	Purpose
GET	retrieve data
POST	create data
PUT / PATCH	update data
DELETE	remove data

Q04 Intermediate

What do the HTTP status code ranges mean?

Range	Meaning
2xx	success (200, 201)
3xx	redirection
4xx	client error (400, 401, 404)
5xx	server error (500, 503)

© 2026 ByteMart Digital

Q05 Intermediate

What would you check in an API response?

- **Status code** — the correct one for the request.
- **Response body** — correct data, structure, and types.
- **Headers** — content-type, auth, caching.
- **Response time** — within acceptable limits.

Q06 Basic

What is Postman used for?

Postman is a popular tool for manually testing APIs — building and sending HTTP requests, inspecting responses, saving collections, and organising tests without writing code.

Q07 Advanced

What is the difference between 401 and 403 status codes?

- **401 Unauthorized** — you're not authenticated (not logged in / bad credentials).
- **403 Forbidden** — you're authenticated but not allowed to access this resource.

Q08 Basic

What is database testing?

Database testing verifies that data is stored, retrieved, updated, and deleted correctly — checking data integrity, CRUD operations, constraints, and consistency between the UI and the underlying data.

Q09 Intermediate

Why verify data in the database after a UI action?

The UI can show 'success' while the database quietly stores the wrong thing. Querying the database directly confirms the data behind the interface is actually correct — a check the UI alone can't provide.

Q10 Intermediate

Write a SQL query to verify a user was created.

```sql
SELECT COUNT(*) FROM users WHERE email = 'sam@x.com';
-- expected: 1

SELECT name, role, active FROM users
WHERE email = 'sam@x.com';
```

Q11 Advanced

What is the difference between DELETE, TRUNCATE, and DROP?

- **DELETE** — removes selected rows (can use WHERE; can roll back).
- **TRUNCATE** — removes all rows fast, keeps the table structure.
- **DROP** — removes the entire table, structure and all.

Q12 Intermediate

What is a JOIN in SQL?

A **JOIN** combines rows from two or more tables based on a related column. Types include INNER (matching rows only), LEFT (all left rows plus matches), and RIGHT. Testers use joins to verify related data across tables.

Q13 Advanced

What negative cases would you test on an API?

- Missing required fields → expect 400.
- Invalid data types or formats → expect 400.
- No / invalid authentication → expect 401.
- Accessing another user's data → expect 403.
- Non-existent resource → expect 404.

© 2026 ByteMart Digital

PART 09

Agile & Test Management

Testing in Agile teams, metrics, and management tools.

12 questions

09

Q01 Basic

How does testing work in Agile?

In Agile, testing is **continuous** — done every sprint rather than as a final phase. Testers are embedded in the team, clarify acceptance criteria, test features as they're built, and automate regression to keep pace with rapid change.

Q02 Basic

What is a user story?

A **user story** captures a requirement from the user's perspective: 'As a [user], I want [goal] so that [benefit].' It's the unit of work in Agile, with acceptance criteria defining when it's done.

Q03 Intermediate

What are acceptance criteria?

Acceptance criteria are the specific, testable conditions a user story must satisfy to be considered complete. Testers use them to derive test cases and confirm the story meets its intent.

Q04 Intermediate

What is the 'definition of done'?

The **definition of done** is a shared checklist a work item must meet to be complete — typically including 'coded, reviewed, tested, and documented.' It ensures 'done' means the same thing to everyone.

Q05 Basic

What is a sprint?

A **sprint** is a short, fixed time-box (usually 1–4 weeks) in which an Agile team completes a set of work. Testing happens throughout the sprint, not after it.

Q06 Intermediate

What is the role of a tester in a scrum team?

- Clarify acceptance criteria during planning.
- Test features as they're completed within the sprint.

- Collaborate closely with developers and the product owner.
- Maintain and run regression (often automated).

Q07 Intermediate

What are some useful test metrics?

- **Test case pass rate** — how much is working.
- **Defect density** — defects per module/size.
- **Defect leakage** — bugs that reached production.
- **Test coverage** — how much is tested.

Q08 Advanced

What is risk-based testing?

Risk-based testing prioritises testing effort by the probability and impact of failure — testing the highest-risk, highest-value features most thoroughly. It's how teams test smartly when time is limited.

Q09 Basic

Which test management tools have you used?

- **Jira** — issue/defect tracking and Agile boards.
- **TestRail / Zephyr** — organising test cases and runs.
- **Confluence** — test plans and documentation.
- **Bugzilla** — dedicated defect tracking.

Q10 Advanced

Why can metrics be misleading?

A metric worshipped becomes a target that's gamed. A high pass rate means little with low coverage; rewarding 'bugs found' encourages logging trivia. Use metrics to ask better questions, not to rank people.

Q11 Intermediate

What is a burndown chart?

A **burndown chart** tracks remaining work against time in a sprint. A steady downward line shows healthy progress; a flat line signals blockers — useful for spotting whether testing is keeping pace.

© 2026 ByteMart Digital

Q12 Advanced

How do you handle a situation where there's not enough time to test everything?

- Prioritise by risk — test critical, high-impact features first.
- Focus on the most-used user journeys.
- Run a targeted regression suite over exhaustive testing.
- Communicate clearly what was and wasn't tested, and the risks.

PART 10

Scenario & Practical Questions

The hands-on 'how would you test...' questions interviewers love.

12 questions

Q01 Basic

How would you write test cases for a login page?

- **Positive** — valid email + valid password → logs in.
- **Negative** — wrong password, wrong email, empty fields.
- **Boundary** — min/max length credentials.
- **Security** — SQL injection, masked password, lockout after N attempts.
- **UI** — field labels, error messages, remember-me, forgot-password link.

Q02 Intermediate

How would you test an ATM machine?

- **Functional** — withdraw, deposit, balance, PIN change.
- **Boundary** — withdraw exactly the balance, over the limit, min amount.
- **Negative** — wrong PIN (and lockout), insufficient funds, invalid amount.
- **Non-functional** — card stuck, power failure mid-transaction, network loss.

Q03 Basic

How would you test a pen?

A classic to test structured thinking. Cover **functional** (does it write, on paper/other surfaces?), **non-functional** (comfort, durability, ink lifespan), **usability** (grip, cap), and **negative** (drop it, leave it uncapped, extreme temperatures).

TIP
The interviewer wants to see you organise testing into categories — functional, non-functional, boundary, negative — not an exhaustive list. Structure matters more than quantity.

Q04 Intermediate

How would you test a text/search box?

- Valid input returns correct results.
- Empty search, whitespace-only, very long input.
- Special characters, SQL/script injection.
- Case sensitivity, partial matches, no-results message.
- Performance with large result sets.

© 2026 ByteMart Digital

Q05 Intermediate

How would you test a file upload feature?

- Valid file types and sizes upload successfully.
- Invalid types and oversized files are rejected clearly.
- Empty file, corrupted file, zero-byte file.
- Multiple/concurrent uploads; cancel mid-upload.
- Security — executable files, malicious filenames.

Q06 Advanced

How would you test an e-commerce checkout?

- Add/remove items, quantity changes, totals and taxes.
- Discount codes — valid, expired, invalid.
- Payment — success, failure, timeout, declined card.
- Boundary — empty cart, max quantity, exact-limit totals.
- Order confirmation, email, and inventory update.

Q07 Advanced

A page loads correctly for you but a user reports it's broken. How do you investigate?

- Gather details — browser, OS, device, steps, screenshot.
- Reproduce on their environment (cross-browser/device).
- Check console logs, network, and cache/cookies.
- Verify data state and any user-specific configuration.

Q08 Advanced

How do you test a feature that depends on a third-party API that's down?

- Use mocks/stubs to simulate the API's responses.
- Test how your app handles the API being unavailable (error handling, timeouts, fallbacks).
- Verify graceful degradation, not a crash.

© 2026 ByteMart Digital

Q09 Intermediate

You found a defect just before release. What do you do?

Log it clearly with reproduction steps, assess its severity and priority, and escalate to the team/lead. The go/no-go decision is a business call — your job is to give a clear, honest picture of the risk so an informed decision can be made.

Q10 Intermediate

How would you test a lift (elevator)?

- **Functional** — call, floor buttons, door open/close, correct stops.
- **Boundary** — max weight/capacity, top and bottom floors.
- **Negative** — overload, obstructed door, all buttons at once.
- **Non-functional** — power failure, emergency stop/alarm, speed.

Q11 Advanced

How would you test a mobile app's login compared to a web login?

- All the web cases (valid/invalid/boundary/security).
- Plus mobile: biometric login, interruptions (call mid-login), rotation.
- Network variability — slow, offline, switching wifi/data.
- App backgrounding and resuming during login.

Q12 Advanced

What's your approach when requirements keep changing?

Stay flexible and communicate constantly. Keep test cases modular and easy to update, prioritise by current risk, maintain a traceability link to the latest requirements, and confirm changes with the product owner rather than assuming.

© 2026 ByteMart Digital